

Report Quiz 2 Design & Analysis of Algorithms

Name

- Arya Rangga Putra Pratama (5025241072) (EF234405 Design & Analysis of Algorithms (D))
- Abrar Maulana (5025241125) (EF234405 Design & Analysis of Algorithms (F))
- Muhammad Cholaif Al Ghifari H (5025241210) (EF234405 Design & Analysis of Algorithms (F))

Class

EF234405 Design & Analysis of Algorithms

Design:

1. Graph Representation

The SurabayaRoute application models the city of Surabaya's tourism landscape as a weighted undirected graph. Each node represents a significant landmark or tourist destination, while edges represent the road connections between them. To ensure real-world accuracy, the edge weights are not based on straight-line distances but on actual road distances obtained via the OSRM (Open Source Routing Machine) API.

- **Vertices (V):** 18 predefined locations (Tugu Pahlawan, House of Sampoerna, Kenjeran Park).
- **Edges (E):** Road networks connecting these locations.
- **Data Structure:** An Adjacency List is utilized to store the graph, providing an efficient balance between memory usage and traversal speed.

2. Algorithmic Choice

The project implements three fundamental graph algorithms to cater to different user needs:

- **Dijkstra's Algorithm:** Implemented using a Min-Heap Priority Queue to find the shortest path in terms of total distance (kilometers). It ensures optimality for travelers prioritizing fuel efficiency and time.
- **Breadth-First Search (BFS):** Utilized to find the path with the minimum number of "hops" or stops. This is ideal for users who prefer fewer transfers or transitions between major nodes.
- **Kruskal's Algorithm:** Used to generate a Minimum Spanning Tree (MST). This feature demonstrates how to interconnect all tourist spots with the minimum total infrastructure length, useful for urban planning or network design.

Implementation:

1. Technology Stack

The application is a web-based interactive dashboard built with HTML5, CSS3, and Vanilla JavaScript. The map interface is powered by Leaflet.js, integrating OpenStreetMap tiles for a rich, interactive geographical experience.

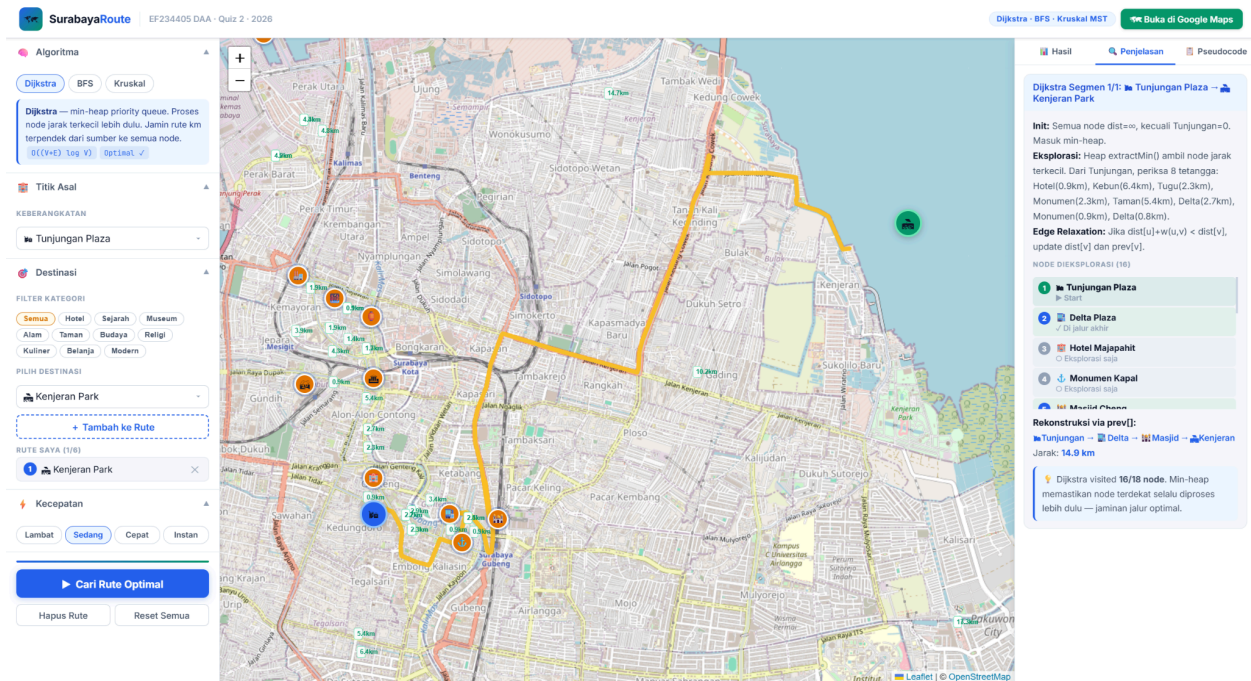
- **Dynamic Theme Switching (Dark Mode):** A comprehensive Dark Mode feature that toggles the entire UI, including custom map tile filtering (brightness/contrast adjustment) to ensure high visibility and reduced eye strain during nighttime usage.
- **State Persistence:** The system automatically remembers the user's login status and preferred theme even after a page refresh, providing a seamless user experience similar to production-grade applications.
- **Dynamic Graph Weighting (OSRM Integration):** Unlike static simulators, this application dynamically fetches real-world road data during initialization and updates the internal graph's edge weights in real-time. This ensures that the Dijkstra and Kruskal algorithms operate on actual road distances rather than simple straight-line approximations.

2. Key Features

- **Interactive Map:** Users can select starting points and add multiple destinations. The map visualizes the pathfinding process in real-time.
- **Algorithm Simulation:** Includes a speed control feature (Slow to Instant) and real-time pseudocode highlighting to assist in educational understanding.
- **Real-time Analytics:** A dedicated panel displays execution time, total distance, nodes visited, and the optimal sequence of steps.
- **Authentication:** A simulated login module ensures secure access to the dashboard, enhancing the "application-ready" feel of the project.
- **Google Maps Integration:** Allows users to export the calculated route directly to the official Google Maps app for navigation.
- **Responsive Layout:** The UI is built using a mobile-first approach with CSS Flexbox and Media Queries, ensuring the dashboard remains functional on both desktop monitors and mobile devices.

Evaluation:

The performance and behavior of the implemented algorithms were tested using various scenarios. The most prominent evaluation involved finding a route from Tunjungan Plaza to Kenjeran Park.



	Dijkstra’s Algorithm	BFS
Total Distance	Optimal (Shortest km)	Sub-optimal
Minimum Hops	Depends on weights	Guaranteed minimum nodes
Execution Time	-150 - 200 ms	-100-150 ms
Node Exploration	Greedy	Layer-by-layer

Findings:

- Dijkstra is highly efficient for distance-based navigation but requires slightly more computational overhead due to the priority queue management.
- BFS is faster for unweighted scenarios but can lead to significantly longer physical travel distances in a city-scale graph where weights vary greatly.
- Kruskal successfully connected all 18 nodes using exactly 17 edges, achieving the minimum possible total distance to cover all points in the network.

Conclusion:

The SurabayaRoute project successfully integrates complex graph theory into a user-friendly, real-world application. By combining theoretical algorithms with modern web technologies and real-world map data, we have demonstrated that efficient pathfinding is essential for smart city initiatives.

- **Work percentage and contributions**

Name	NRP	Percentage	Contributions
Arya Rangga Putra Pratama	5025241072	35%	Designed the user interface and implemented Dijkstra, BFS, and Kruskal algorithms.
Abrar Maulana	5025241125	35%	Handled backend logic and developed the main application structure.
Muhammad Cholaif Al Ghifari H	5025241210	30%	Performed analysis, testing, and authored the final report.